

1 Caesar and Simple Cryptanalysis

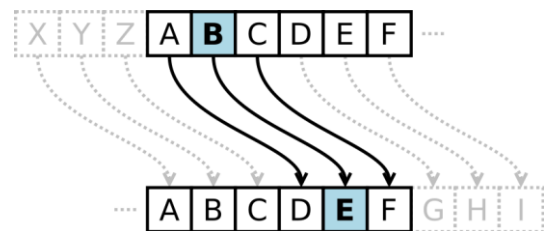
To complete the tutorial, you need to work with a friend on two machines. You will perform the bulk of the work together on one machine and then perform the signature verification on the other machine.

2 Caesar Encryption

In this lesson, we will perform a simple classic encryption using the Caesar algorithm and then subsequently perform a cryptanalysis on the encrypted document and calculate from there the key so as to be able to decrypt it.

In cryptography, a Caesar cipher, also known as a Caesar's cipher, the shift cipher, Caesar's code or Caesar shift, is one of the simplest and most widely known encryption techniques. It is a type of substitution cipher in which each letter in the plaintext is replaced by a letter some fixed number of positions down the alphabet. For example, with a shift of 3, A would be replaced by D, B would become E, and so on. The method is named after Julius Caesar, who used it to communicate with his generals.

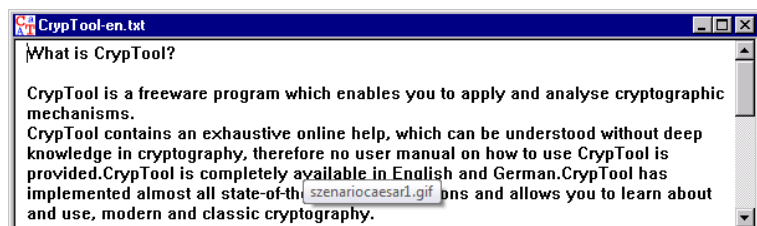
The encryption step performed by a Caesar cipher is often incorporated as part of more complex schemes, such as the Vigenère cipher, and still has modern application in the ROT13 system. As with all single alphabet substitution ciphers, the Caesar cipher is easily broken and in practice offers essentially no communication security



(http://en.wikipedia.org/wiki/Caesar_cipher)

To acquaint ourselves with encryption, we will explore a very simple encryption algorithm - the Caesar encryption algorithm. We will open a document, encrypt it and then decrypt it again. We will then try to get the computer to work out the key with which a plaintext is encrypted.

**2.1 Open the file
CrypTool-en.txt
out of the
directory:**



CrypTool\examples via the menu File→Open.

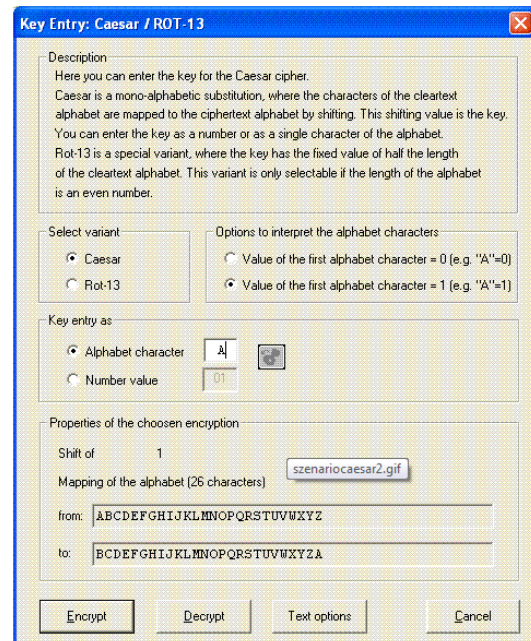
We will now encrypt this document using the Caesar encryption algorithm

2.2 To do this, select the menu **Crypt/Decrypt→Symmetric (classic)→Caesar/Rot-13**

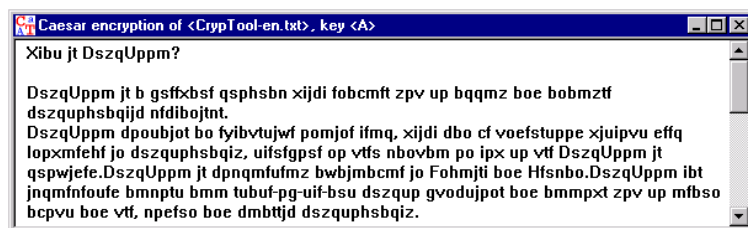
2.3 The key we enter is the letter **A**

2.4 Additionally, we change the *options to “interpret the alphabet characters”*: The first alphabet character is set to 1 (if we set the first character to 0 and use A as key, then the resulting ciphertext is the same as the plaintext).

2.5 Click on the **Encrypt** button

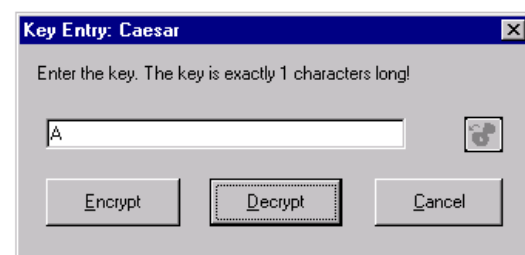


This opens a new window that contains the encrypted text. On closer examination of the text it becomes apparent that the letters have been shifted by one position, so that the initial word, **CrypTool**, is now **DszqUppm**.

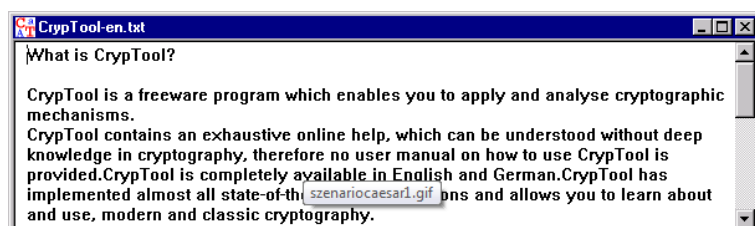


2.6 The plaintext version of this encrypted document can now be obtained by selecting **Crypt/Decrypt→Classical→Caesar** again

2.7 In the dialog box which now appears we enter the key with which the document was encrypted (**A**)



2.8 This time we do not want the text to be encrypted, but instead to be decrypted. Therefore the **Decrypt** button must be selected.



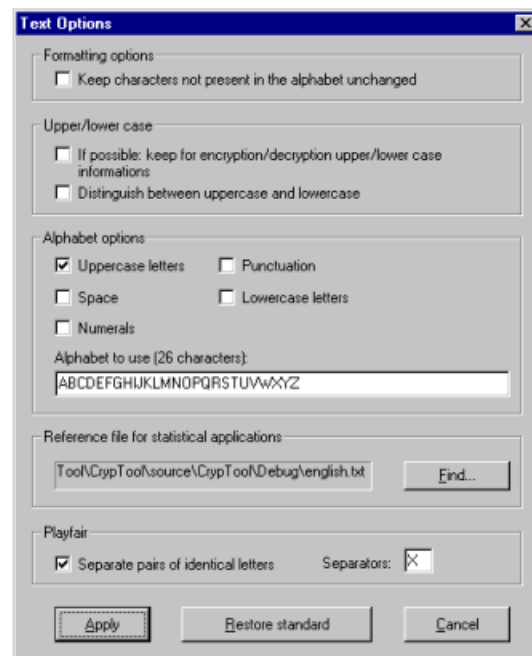
The plaintext appears immediately.

Under the classical Caesar encryption algorithm only the letters are encrypted. During encryption, the lower case letters were converted to upper case letters and encrypted, but all the other characters such as punctuation characters and formatting characters (blank characters and line breaks) were omitted.

In the classical encryption algorithms CrypTool retains the formatting by default.

2.9 This can be disabled via the menu option Options→Text Options.

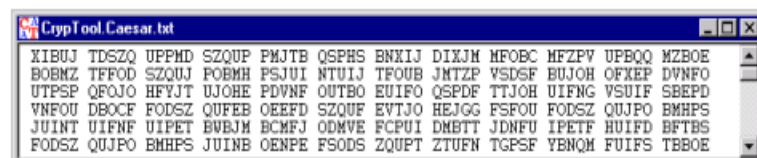
In the following dialog box the option Keep characters not present in the alphabet unchanged and the option Distinguish between uppercase and lowercase must be disabled.



2.10 Return to the window containing the plaintext (by clicking on it with the mouse)

2.11 Encrypt this document once again with the Caesar encryption algorithm, using the key A

From the results one can see immediately that the encrypted text has the same content as the



previous version but this time it contains only upper case letters and the formatting has been removed. For example, the first characters in the encrypted text are once again **DSZQUPPM** (the ciphertext of CrypTool).

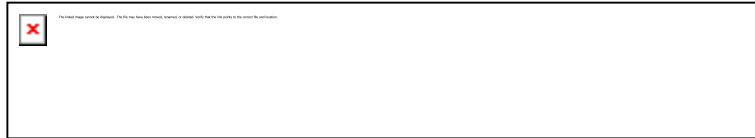
To improve legibility, blocks of five characters have been created and a space has been inserted after every fifth character.

We would like to save this document under a different name.

2.12 Select File→Save As to access the dialog box used for saving a document under a new name

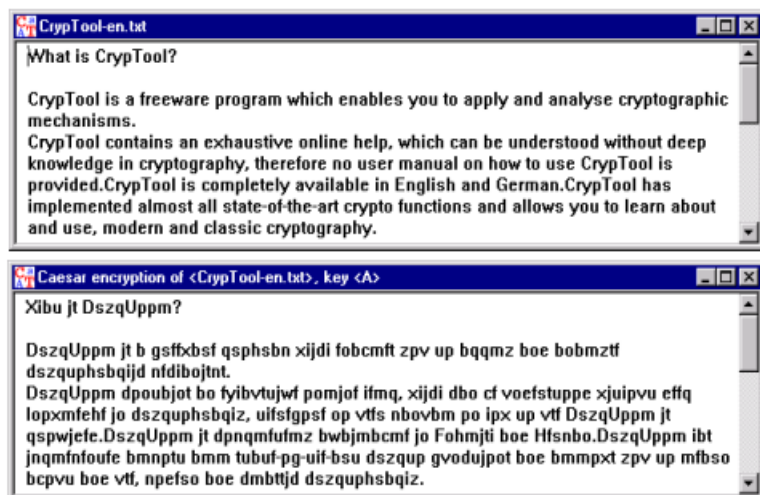
2.13 Save the file as CrypTool.Caesar.txt

The decryption operation works in exactly the same way as described above. We decrypt this document once again and the plaintext version is restored. This of course now consists only of a sequence of upper case letters. Because no distinction is made any longer between upper and lower case letters, and especially because of the absence of formatting, this text is more difficult to read. However, if we compare the document briefly with the original text, we can see that it is the same text. The original text begins with the words, "CrypTool is a program which will enable you ...". These words appear once again in the first line of the decrypted text.



Having now learned the steps involved in encrypting and decrypting a document, we would like to take a look at the security of the Caesar encryption algorithm.

2.14 To do this, all the windows should be closed apart from the window containing the plaintext and the version of the encrypted text in which the formatting is retained. As shown in the figures.



2.15 First, make the window containing the plaintext the active window again by clicking on it with the mouse

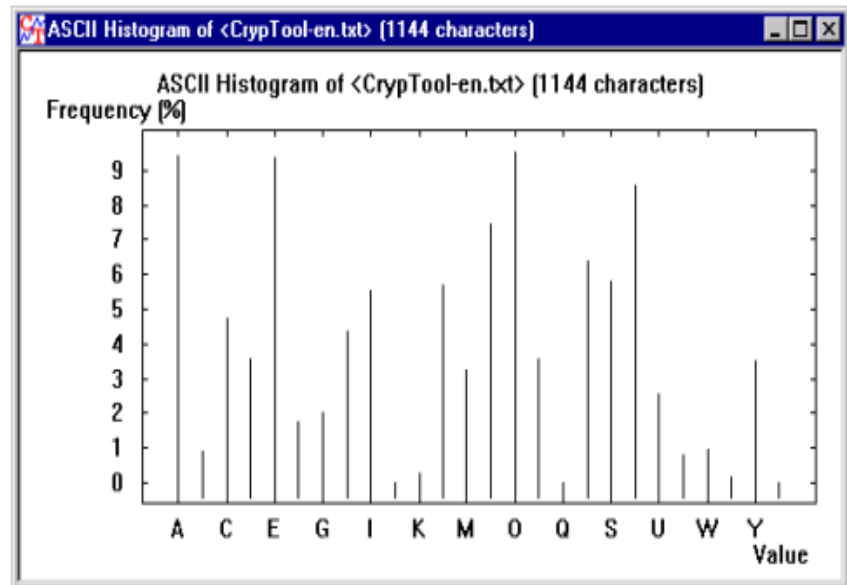
We are now going to conduct an analysis of this document. This is sometimes useful to do on encrypted text as part of the process of cryptanalysis. More information on the use of entropy in cryptanalysis can be found in the following article: <http://www.adacrypt.com/downloads/Entropy%20and%20Structure%20in%20Cryptography.pdf>

2.16 Access the Analysis→Tools For Analysis→Entropy menu



2.17 Next we analyse the frequency distribution of the letters (Analysis→Tools for Analysis→Histogram).

2.18 Repeat the same procedure after making the other window active



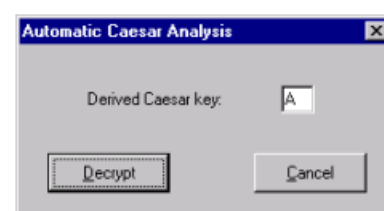
We can see immediately that both documents have the same entropy.

When we examine the histogram of the encrypted document we can see that the letter frequencies have merely been shifted by one position. That means that this old encryption algorithm is not secure. (Moreover there are only 26 possible keys, and of these the Z shifts the text by 26 positions so that the "encrypted" text is actually identical with the unencrypted text. Hence there are really only 25 possible keys!)

The Caesar encryption algorithm can be broken easily by a ciphertext-only attack.

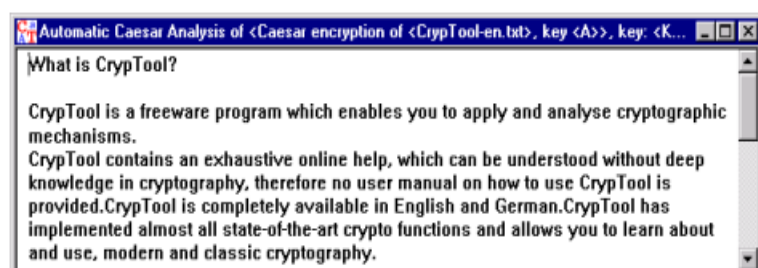
2.19 To perform such an attack, restore the window containing the encrypted text to the active window and select Analysis→Symmetric Encryption (Classic)→Ciphertext-only→Caesar

This will analyse the text, generate two histograms and also identify the key as being 'A'



2.20 Select Decrypt

When you click on the Decrypt button in the message window, the plaintext appears, i.e. the text that has been decrypted with the key A that was discovered.



3 'Classic Encryption' - Vignere

A modification on this is the Vigenere cipher. This algorithm relies heavily on a long key phrase. Letters from the ciphertext are matched up to the key phrase and then substituted to the corresponding character.

Hence if the key/password is:

password

and the message is:

samplemessage

(one word), the 's' in the

vertical column is traced across to the 'p' in the horizontal row. The intersection of the two is 'h', hence the character p is substituted for a 'h'. In the same way, the ciphertext in this case becomes:

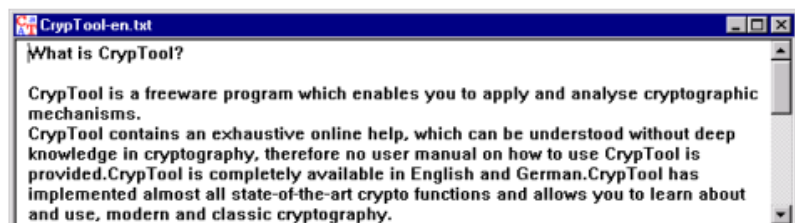
haehhsdhhssya

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

We are going to perform a very quick encryption using the Vignere algorithm.

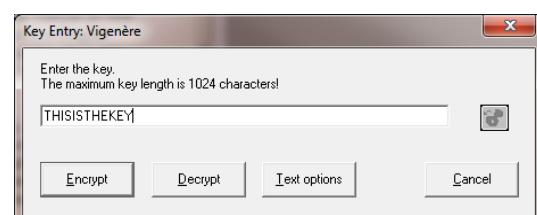
3.1 Open the file **CrypTool-en.txt** out of the directory: **CrypTool\examples** via the menu **File→Open**.

3.2 Encode this document using **Crypt/decrypt→Symmetric(classic)→Vignere**

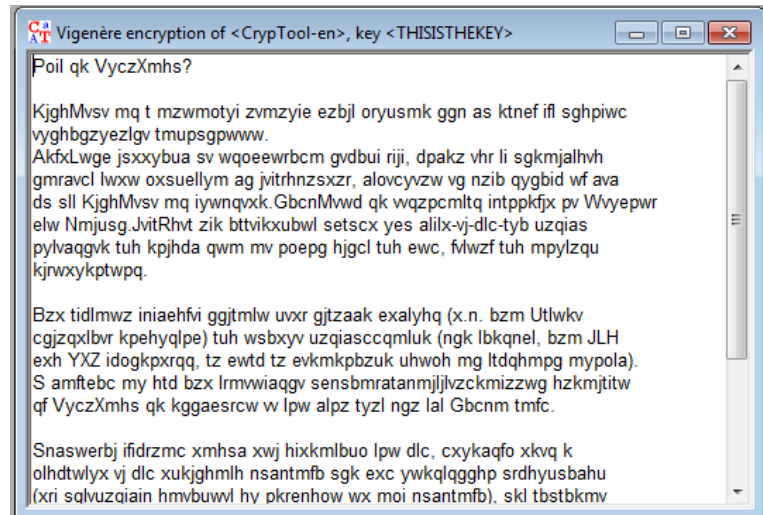


3.3 Enter: **THISISTHEKEY** as the key

3.4 Click on **Encrypt**



This presents the encrypted document



4 Example illustrating encryption using binary addition

In this section we will look at the complexities of performing a cryptanalysis on compressed data. We will use binary addition as the basis for our encryption.

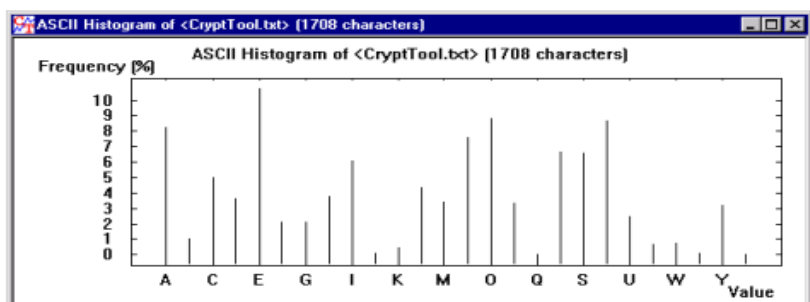
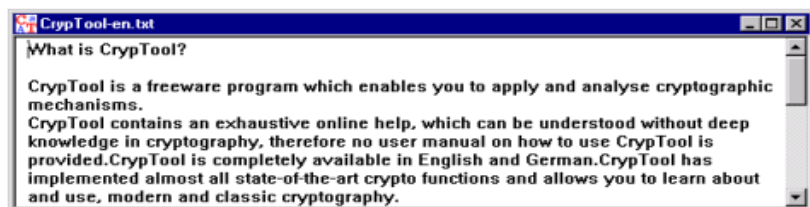
A text document will be used to illustrate encryption through binary addition and a Ciphertext Only attack on this method. It will be shown that with many encrypted documents it is easy to work out the key and hence the plaintext as well. However, if the plaintext is compressed prior to encryption, the key and the plaintext cannot be worked out directly with CrypTool as the frequencies of the individual characters prior to encryption (but after compressing) are not known. On the other hand, certain properties of uncompressed documents, such as texts, are known and can therefore be utilised in the analysis.

The example presented here functions in a manner similar to a binary document containing, for example, a graphic image (in contrast to the text document used in this example). In an example illustrating encryption using the Exclusive-OR, the steps performed are similar to those described here except that the file that is encrypted is a graphic image in the form of a bitmap, i.e. a binary document.

4.1 Open the file CrypTool-en.txt out of the directory: CrypTool\examples via the menu File→Open.

Before the document is encrypted, let us examine the frequency distribution of the characters as presented in the form of a histogram.

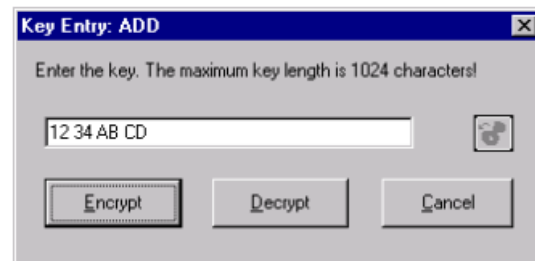
4.2 Select Analysis→Tools for analysis→ Histogram



One can see from the histogram that the character which occurs most frequently is the letter E. This is true of many German and English texts. This information will be utilised later on during our attack.

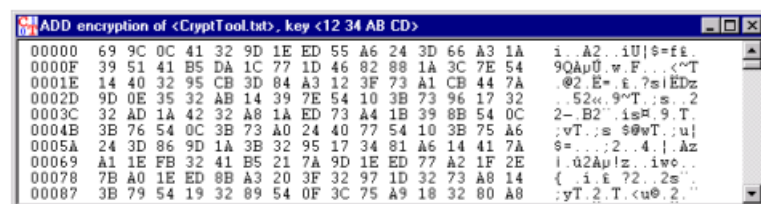
4.3 Make the text document the active window again by clicking on it with the mouse.

4.4 To encrypt the document using binary addition, select **Crypt/Decrypt→Symmetric (classic)→Byte Addition**



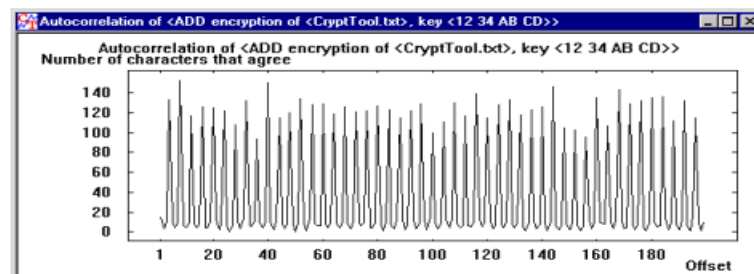
4.5 Enter 12 34 AB CD into the dialog box which opens as the key.

4.6 Click the **Encrypt** button, and a new window opens containing the encrypted document.

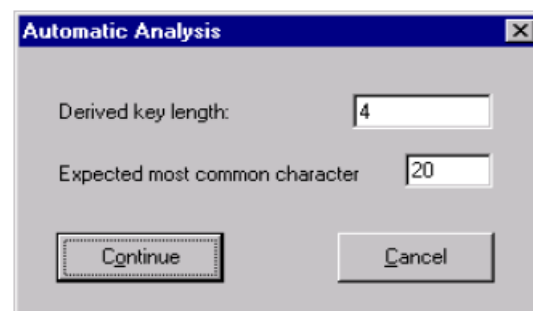


We will now perform a ciphertext-only attack

4.7 Select **Analysis→Symmetric Encryption (classic)→Ciphertext-Only→Byte Addition**, and the autocorrelation will be calculated and displayed.



Following a click on the OK button the Automatic Analysis dialog box appears, in which we are told that the key length is calculated to be 4. As we saw at the beginning, the commonest character is the letter E. The hexadecimal value of E is 45 (see ASCII Table). If, however, we return briefly to the plaintext, we notice that the character which occurs most frequently is not the upper case letter E, but the lower case letter e. In most texts, the lower case letter e is the character which occurs most frequently. In shorter texts it could also be the space, which has a hexadecimal value of 20. The hexadecimal value of e is 65.



4.8 Enter 20 into the dialogue box and press continue

CrypTool has been able to find the key, as can be seen from the next box. The only information that was needed to do this was the fact that the character which occurred most frequently in the plaintext was the lower case letter e.



4.9 Click decrypt, this produces a new window containing the plaintext.



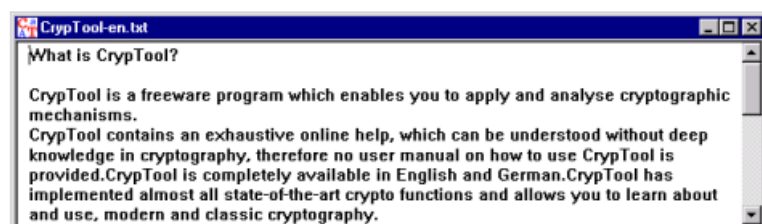
The key 12 34 AB CD, with which the text document was originally encrypted, is displayed in the window title bar.

In this case it was no problem for CrypTool to work out the key and hence the plaintext version of an encrypted document as we did have some information about the encrypted document, namely that it contained text. We also knew that the lower case letter e is often the character that occurs most frequently in textual documents.

But if the text is compressed prior to encryption then we will not be able to draw any conclusions from the frequency distribution of the characters in the text about the frequency distribution of the compressed text, since the compression process not only reduces the size of a file but alters the frequencies of the individual characters so that they no longer reflect the frequencies of the characters in the original text. This means that we do not know the commonest character and cannot help CrypTool with the analysis.

Let's demonstrate this.

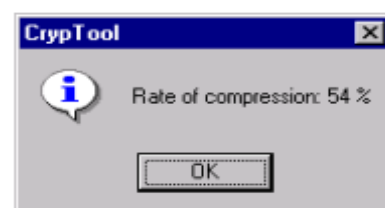
4.10 To compress the document, it must be made the active window again.



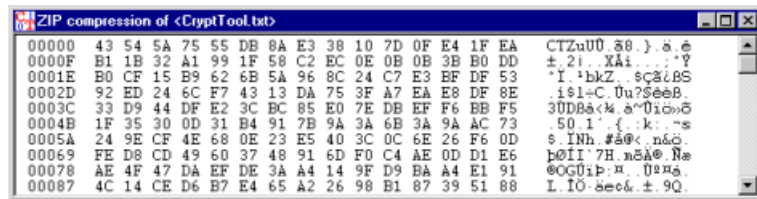
4.11 It can now be compressed by selecting the menu option Individual procedures→tools→Compress→Zip.

The rate of compression is displayed, following which the compressed document is shown.

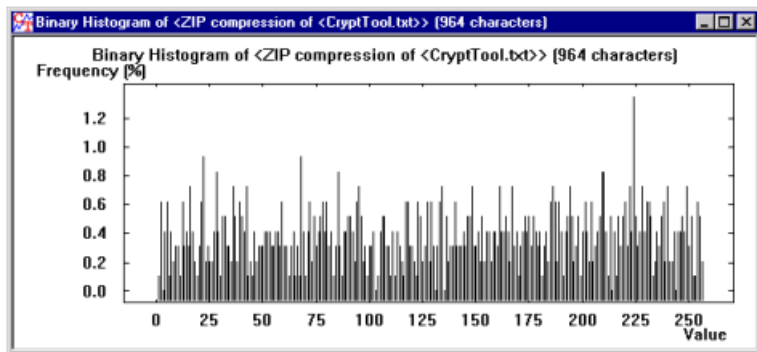
4.12 For this document too, let us first take a look



at the histogram
(Analysis→Tools
for
Analysis→Histogram).



One can see immediately that the compression process produces a quite different histogram profile from the one previously obtained for the uncompressed document. The characters are much

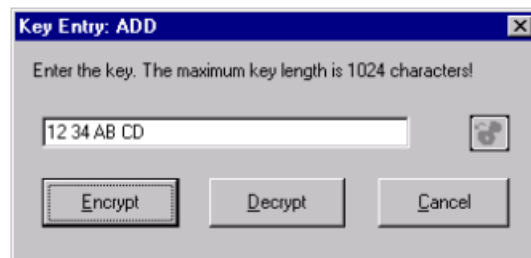


more evenly distributed than in the unencrypted document.

While it is possible to attempt to analyse an encrypted, compressed document, because we do not know which character occurs most frequently in a compressed document, we cannot provide CrypTool with any help during the analysis and must therefore use the default value of 00 as the character which occurs most frequently.

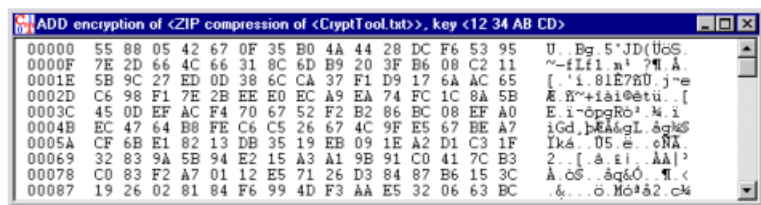
4.13 Make the compressed document the active window again

4.14 Encrypt the document using binary addition, select Crypt/Decrypt→Symmetric (classic)→Byte Addition



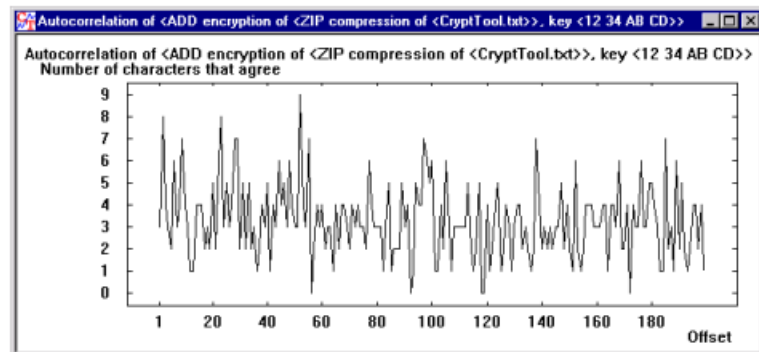
4.15 Enter 12 34 AB CD into the dialog box which opens as the key.

Following input of the key and a click on the OK button, the window with the encrypted document appears as usual.

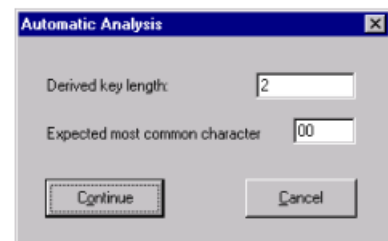


4.16 Select Analysis→Symmetric Encryption (classic)→Ciphertext-Only→Byte Addition, and the autocorrelation will be calculated and displayed.

We now invoke the Analysis function again and see that the autocorrelation process has not been as accurate as before in its estimate of the length of the key as before.

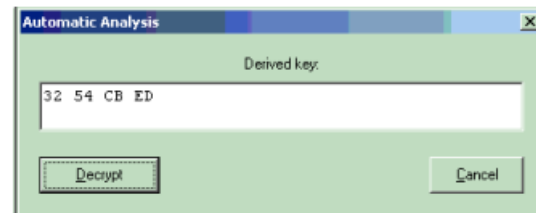


CrypTool in fact returns an incorrect key length and an incorrect derived key. Given this key length, it is not possible to find the correct key either.



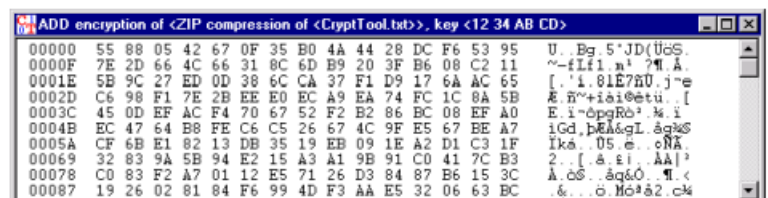
4.17 Click Continue and in the next dialogue box select decrypt

This displays an incorrect version of the plaintext document. This document bears no similarity with the original compressed, but unencrypted document.



Finally, we would like to check whether it is possible to arrive at a readable version of the text document from the compressed and then encrypted document. To do this we need to know the key with which the document was encrypted (12 34 AB CD) and the encryption algorithm (binary addition).

4.18 First of all we make the compressed, encrypted document the active window again.

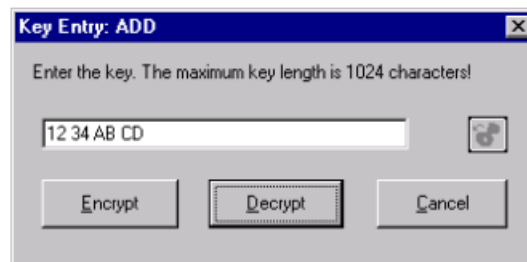


We now decrypt the document with the known key

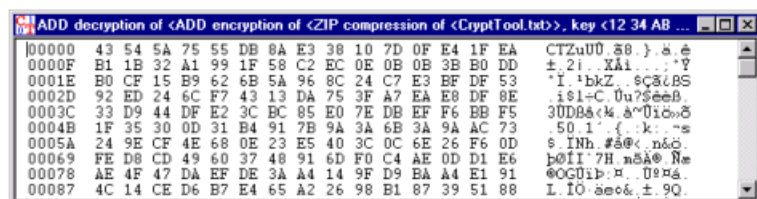
**4.19 Encrypt the document using binary addition, select
Crypt/Decrypt→Symmetric
(classic)→Byte Addition**

4.20 Enter 1234ABCD as the key

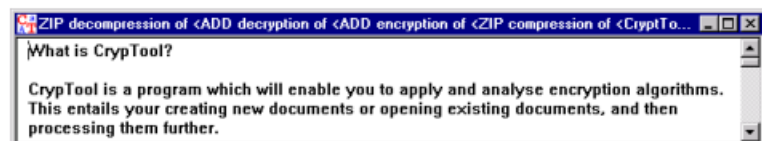
4.21 Select Decrypt



This produces
unencrypted, but still
compressed document.



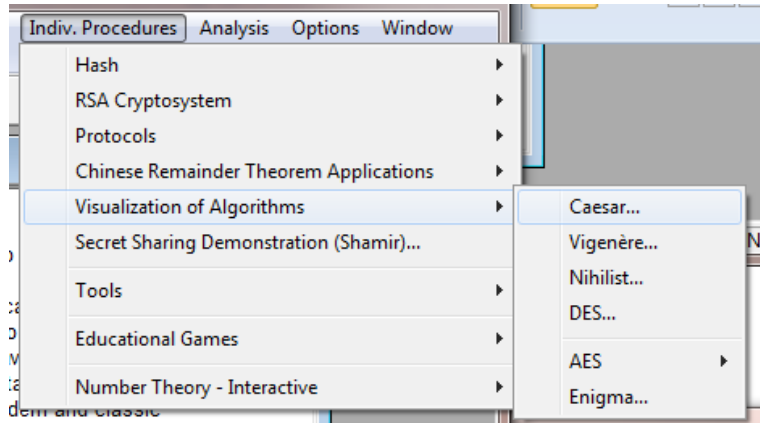
**4.22 It can now be decompressed by selecting the menu option Individual
procedures→tools→
Compress→UnZip.**



5 Caesar Demonstration

Caesar is one of the more popular 'encryption' algorithms (although most would argue it is actually a substitution system) for teaching. The reason for this is that it is 'easy' to understand and forms a sound basis for understanding some fundamental encryption concepts.

CrypTool comes with an interactive simulation of the Caesar cipher which can help you to understand the process a lot better.

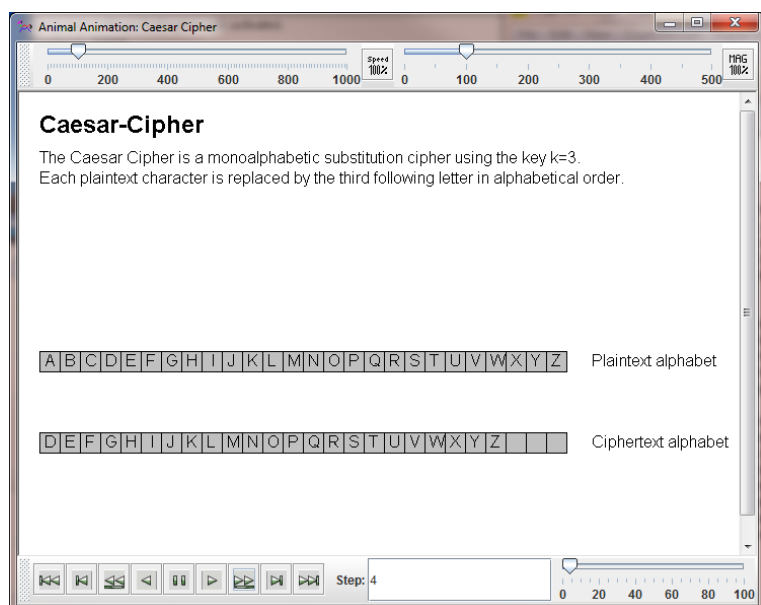


5.1 Access this tutorial by going through the Indiv.Procedures→Vizualisation of Algorithms→Caesar menu

This will launch a simulation system called 'Animal'. If this is the first time it has been launched on your machine, it might take a while to load. Once up and running you need to press the 'play' button to move it forward, you can move it back at anytime using the icons.



5.2 Make your way through the demonstration



6 Task

- 6.1 Perform a Histogram analysis on the document “Startingexample-en.txt”, does this document support the theory of the ‘e’ character being the most popular character in a typical document?
- 6.2 According to this document, what is the second most popular character?
- 6.3 Create a new document in CrypTool (File→New)
- 6.4 Copy and paste the following text into the newly created document:

Mfiuxtma wn lqnnmzmmb sqvla wn ibbiksa

I vcujmz wn lqnnmzmmb bgxma wn ibbiks wv kzgxbwagabmua qa xwaaqjtm. Bpmam ibbiksa zmycqzm lqnnmzmmb tmdmta wn swetmlom, kwuxcbqvo xwemz, ivl idiqtijqtqbg wn libi wv bpm xizb wn bpm ildmzaizg. Pwemdmz, qb qa vwzuittg iaacuml bpib bpm ildmzaizg abizba nzwu bpm xwaqbqvw wn svweqvo epqkp mvkzgbqvw itowzqbpu qa jmqvo caml. Bpqa iaacuxbqvw pia xzwdml kwzzmkb qv xzibqkm, ia qb pia xzwdml dqzbcittg quxwaaqjtm bw smmx mvkzgbqvw itowzqbpu epqkp izm vwzuittg quxtmumvbm qv nzmmtg idiqtijtm xzwoziua amkzmb qv bpm twvo-bmzu.

Bpm kqxpmbmb-wvtg ibbiks pia bpm tmiab zmycqzmumvba nwz bpm ibbiksmz'a swetmlom – kwvaqlmzqvo bpm swetmlom wn bpm mvkzgbml bmbf ivl bpm xtiqvbmbf. Bpm ibbiksmz'a lmuivl ziqam bpmv nzwu i svwev-xtiqvbmbf ibbiks bw bpm kpwamv-xtiqvbmbf ibbiks.

Bpm mnnwzb zmycqzml bw xmzmbzibm bpm ibbiks qa qvdmzamtg xzwxwbqvwit bw bpm zmycqzmumvba: Eqbp i svwev-xtiqvbmbf ibbiks bpm ildmzaizg pia wvm amb wn xtiqvbmbf ivl mvkzgbml bmbf idiqtijtm, epmzmia qv bpm kqxpmbmb-wvtg ibbiks pm pia wvtg bpm mvkzgbml bmbf. Pmvkm i kqxpmbmb-wvtg ibbiks qa omvmzittg uwzm lqnnqkctb bpiv i svwev-xtiqvbmbf ibbiks. Qv i kpwamv-xtiqvbmbf ibbiks, wvm kiv omvmzibm ivg vcujmz wn amba wn xtiqvbmbf ivl mvkzgbml bmbf, pmvkm i kpwamv-xtiqvbmbf ibbiks bisma tmaa bqum bpiv i svwev-xtiqvbmbf ibbiks.

Ivwbpmz bgxm wn ibbiks qa bpm jzcbm-nwzkm ibbiks. Qv bpqa bgxm wn ibbiks, iv ibbmuxb qa uilm bw omb bpm kwzzmkb smg jg bzgqvo wcb mdmzg xwaaqjtm xmzucbibqvw (nzwu eqbpqv i xzm-axmkqnqml zivom). I dizqibqvw wn jzcbm-nwzkm ibbiksa izm lqkbqvwizg ibbiksa wv xiaaewzla.

Wbpmz bgxma wn ibbiksa izm:

* uiv-qv-bpm-uqlltm ibbiksa

* aqlm-kpivvmt ibbiksa

KzgxBwwt kwvbiqva mfiuxtma qv epqkp bpm mfqabqvo ibbiksa wv mvkzgbqvw itowzqbpu kwvbiqvm qv KzgxBwwt izm Imakzqjml.

- 6.5 Perform a cryptanalysis on this document. What is the first line?